QCon
全球软件开发大会

InfoQ 技术大会

当验证追上生成

AI Native 质量保障的体系化实践

专题 · Vibe Coding 时代的新质量债

李桐馨

飞猪 · 测试开发



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 为什么需要让验证追上生成

02 AI Native 测试如何跑成全链路闭环

03 阶段三：AI 深入之后的新问题

01

为什么需要让验证追上生成

生成并行持续扩大，验证承载仍受人工资源约束

生成可以并行，材料却更难人工评估

AI 产出：可并行扩展

生成侧

并行生成

产品AI · 产出需求

开发AI · 页面 / 接口 / 代码

→ 同时进入

材料更多、更分散

页面 / 接口 / 状态 / 异常分支 / 替代路径

散落在 需求 / 技术方案 / 代码改动 之中

AI产出 —— 随并行度水涨船高 ↑

VS

人工承接：不能并行扩展

评估侧

每个需求、每份变更要压着四个判断：

- ? 需求说清了吗
- ? 方案可验证吗
- ? 影响什么存量
- ? 应该怎么验证

这四个判断，只能人工，一次一条地做。

人工串行 —— 承接容量基本是平的 →

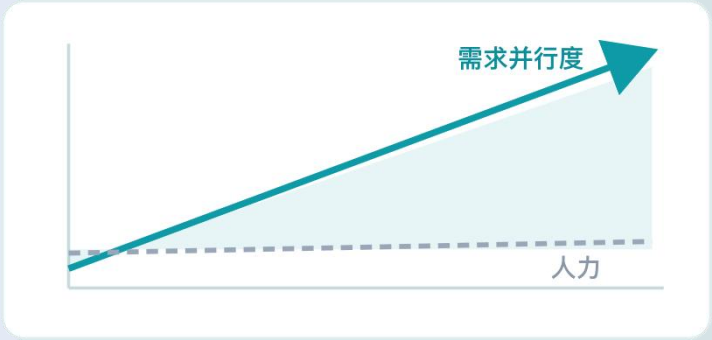
为什么传统（人工）测试接不住

1 评估只能人工串行



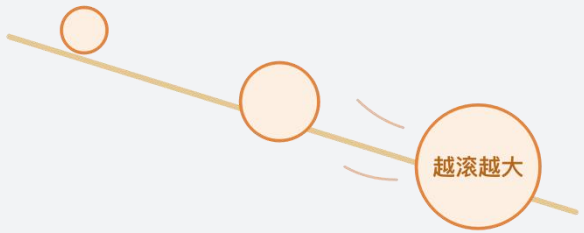
理解材料、追问缺口、判断影响、设计验证。
四个判断逐条人工 —— 无法并行。

2 质量建设无法同步扩展



自动化录入、线上回归监控、基线维护，
都靠人抽时间 —— 跟不上并行量。

3 存量风险滚雪球



变更并行推进，覆盖与验证的欠账累积，
后续判断成本与风险 越滚越大。

面对并行生成和材料复杂度上升，传统以人工为主的测试难以承接——这部分积累下来的不确定性，就是 **验证债**

02

AI Native 测试如何跑成全链路闭环

从AI进行测试环节提效，到逐段递进验证的闭环

阶段一：用 AI 给每个测试环节提效

1 做法 | 把测试拆成环节，每个环节用 AI 顶上——单看，每一步都提效了

用例生成 一天 → 20min 内
AI 读 PRD·技术方案·知识库自动生成用例

→

接口测试 调用 + 断言全自动
AI 生成接口调用与断言并执行

→

UI 测试 真机自主执行
本地 Agent 连真机，自主操作与校验

2 问题 | 快是快了，但产出不可信 / 不稳定 / 上不了规模

源头 · 可信 问题一

生成不稳：质量取决于源头材料

用例生成快了，但文档说不清 / 知识没沉淀时，缺断言、漏场景，甚至凭空假设。

生成采纳率 仅 ~50%

结论 · 可信 问题二

假性成功：绿灯是假的

返回码正常 / 页面能开就判过，真缺陷被放过；同一 case 还忽过忽败。

返回 200 ≠ 验证到

执行 · 规模 问题三

单条跑得慢，还看描述清不清

描述含糊 AI 就跑偏，或跑通了却没验到关键点；单条执行比人工还慢。

单页面点击 AI ≥30min · 人工 2min

3 结论 | 提效没把人省下来，只是把时间从执行挪到了确认

每步都更快了，但一个需求的人工总耗时并没有减少

时间只是从「执行」搬到了「确认」—— 产出不可信，省下的执行又被人工确认吃掉

阶段二的解法：沿研发流程逐段递进验证，结构化进、结构化出

以前（阶段一）

需求与方案阶段快速通过

↓

不确定性积累到测试阶段

↓

测试阶段集中暴露

↓

返工

现在（阶段二）

沿研发流程的每一个阶段，消费上一段的结构化输出，校验并产出下一段可消费的结构化输入。

不确定性在哪个阶段注入，就在哪个阶段拦截，不传递给下一段。

核心原则

结构化进、结构化出——每一段不是「做完就交出去」，而是「校验过、结构化过」，下一段才能接。

整体策略：从源头质量校验到执行证据与回流



用例生成与数据构造：从"可生成"升级为"生成即可执行"

用例生成：三条落地经验

1 经验一 接上游

上游已校验，交付格式统一

格式统一的交付包（固定 schema）

- 需求共识 · 已确认决策表
- 交付共识 R1–Rn · 每条挂验证
- 各仓库 CodingPlan
- 设计稿 Figma · 分支 / commit

missing [] 显式列出缺口

缺的 → 阶段零去 CodeWiki 补齐

缺接口协议 → 命中业务域 API 文档

缺历史用例 → 业务域 Wiki 反向召回

2 经验二 提覆盖率与准确率

靠结构和规则，不靠让模型多写

覆盖率 先召回基线、搭骨架，再铺风险

① 不白纸起稿：召回历史基线 + 建功能树

② 按风险清单铺：三态 / 弱网 / 边界 / 降级 / 多端 / 正逆向

③ 叠「多·并·复·异·资」五字法，反查补漏

PRD 没提、前端已实现的页面 → 标出来

准确率 每个预期结果都要对得上原文

禁「显示正常」等空泛 → 文案逐级回填到字段：

设计稿 > 组件默认 > Bridge > 错误码 > PRD

采集不全标【待确认】不编码；脚本 + 四维自审 + 五道过滤

诚实边界： 模糊/冲突标【待确认】、阻塞暂停；生成后仍需测试 review

输出 结构化用例

- 结构化测试用例（字段级预期）
- 自动化决策（接口 / UI 执行判定）
- 数据绑定（语义化内联步骤）

这份输出先交给下游环节

数据构造 （独立环节）

按数据绑定 查数 / 造数 → 收敛数据场景

接口自动化 / UI 飞碟 执行

3 经验三 · 少造数 让多条用例合并共用一个数据场景，别一条一造

大量结构化用例 → 每条带「数据绑定」前置

→ **按数据需求收敛 / 合并** → 同类前置 → 收敛成同一数据场景

→ 少量数据场景 → 一场景的数据构造一次

→ **一次造数，复用给挂在场景下的整组用例** → 造数次数压到最低 —— 收敛与造数发生在「数据构造」环节

案例 A 实证： 87 条用例 → 18 个数据场景，造数降 79%

接口自动化把三件事做「对」：调用准 · 判断细 · 调用安

① 调用准

调用对齐线上真实接口

接口名 / 参数 / 版本可能是 AI 编的，
看着能跑其实不存在。

AI 拟调用
`service · method · 参数类型`

↓ 编译前用线上接口元数据逐条校

✓ 对得上 / pTypes 不一致自动修正

按真实元数据对齐，调用才是真的，
不是「看着能跑」而已。

✗ 查无此接口 / 参数

直接告警，绝不生成假调用——
不是 AI 说有就有。

② 判断细 · 核心

判定编译成字段级 SpEL 断言

返回 200、调用成功 ≠ 结果对——
绿灯可能是假的。

人话预期　第 5 项的 type 应为『新增项』

↓ 编译（确定性翻译，根对象 = 接口响应）

业务断言 · SpEL

```
result.data['list'][4]['type'] == '新增项'
```

● 响应字段路径　● 取第 5 项　● 到具体字段　● 比对期望值

基线 `success==true && code=='200'` 只是前缀，**看这行字段断言定成败**

翻不了的（异步时序、落库校验）**显式标「需人工」、不硬编**

③ 调用安

写接口分级授权

写接口若测试库 / 线上库未隔离，
直调会污染真实用户状态。

写接口不一刀切，看有没有改造
是否已隔离测试库与线上库

↓ 按隔离状态分级授权

✓ 改造过 / 已隔离

才放开 Agent 自动执行，
写操作落在隔离的测试库。

✗ 未改造 / 未隔离

只生成批量测试任务，
由测试确认参数后再执行。

调用准 · 判断细 · 调用安 —— 三件都做对，接口自动化才从「能跑」变成「结论可信、过程安全」。

飞碟平台：从脚本方式升级到 Agent 驱动

升级前 脚本方式

自然语言用例（上游交下来） —✗— 要脚本 + 控件定位（xpath / 录制回放） **读不懂**

- 自然语言无法直接执行，得人工先翻成脚本
- 用例一改脚本就挂，**维护成本高**

升级

升级后 Agent 驱动

用例怎么写就怎么跑

自然语言用例 → **Agent 自规划** → 真机执行 √

- **直接读自然语言用例**，中间不用再落脚本
- 自己规划路径：从哪进、点什么、看到什么算过

⟹ Agent 真机跑带来两个代价 —— 这两条飞碟都得解决，才跑得了量、结论才靠谱

- **单条慢** Agent 比脚本、比人都慢 → 自建设备池多机多批并发摊平
- **结果不稳** Agent 执行忽红忽绿 → 能固定的固定，只把规划交给 AI

UI自动化：飞碟 UI 自动化平台架构

接入层

- Skill 一　本地 Agent 驱动本地设备：新需求测试边写边跑
- Skill 二　用例录入云端：沉淀为后续回归批次

控制台

- 用例与配置
- 批次与执行
- 设备与队列
- Skill 管理
- 工具库管理

平台服务

- DSL 解析归一化
- 任务与批次编排
- 设备调度
- Skill 与工具库治理
- 执行状态机
- 结果与产物存储

执行引擎

- 上下文注入
- Agent 执行循环
- 设备控制命令层
- 环境与账号准备
- 校验与验收
- 失败归因与回传

设备层

- 自建设备池
- 本地设备

支撑能力

业务 Skill 库

页面怎么走、什么算对

平台与业务解耦，按业务线隔离

工具库

通用框架支持 HTTP / Bash / MCP

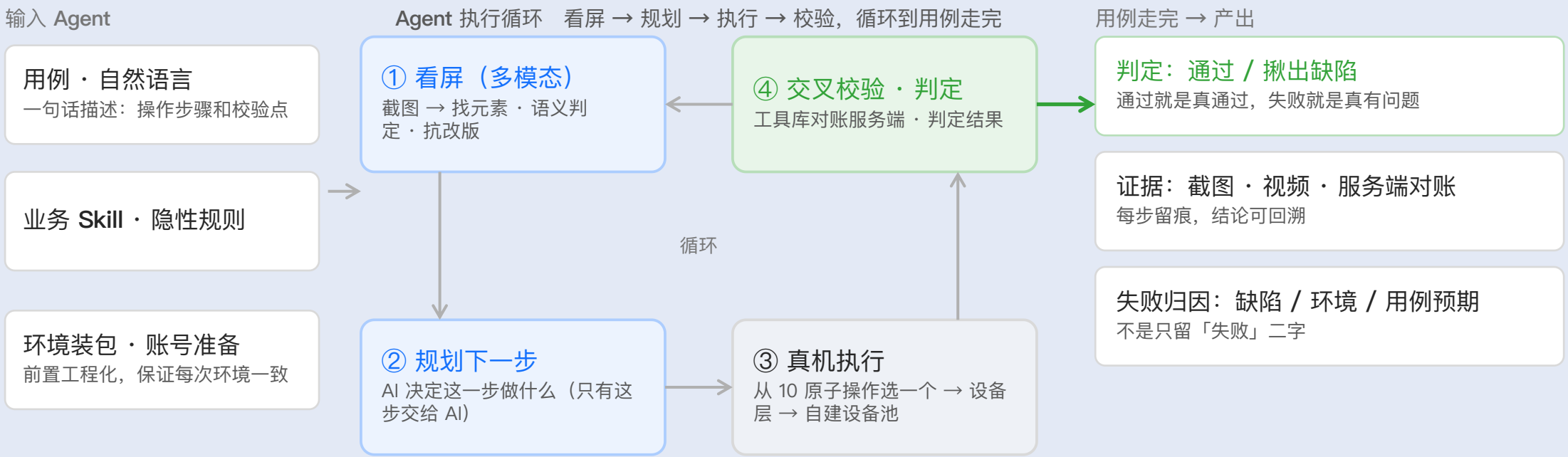
将服务端数据带入验证

多模态模型服务

看截图、找元素、判定结果

支撑视觉与语义校验

UI 自动化 · 怎么做：一条用例在飞碟上的执行机制



稳 · 确定性工程化

③真机执行、环境装包都封死固定，只把②规划交给 AI——同一 case 每次环境一致

准 · 看得懂 + 对得上

①多模态看屏 + ④工具库交叉校验：页面数字去对服务端——例 接口 9950 分 ↔ 页面 ¥99.5，对不上就是假绿灯（端侧版「假性成功」）

UI 执行效果：300+ 条回归发现 7 个真实缺陷

亮点
01 保险辅营 300+条全量回归，通过率 97%左右，发现 7 个真实缺陷
02 云端 6 批并行 ，时间压缩到串行的 1/3 以内
03 补充验证：测新场景的 27 条需求验证，3 条失败全是真实页面缺陷，现有自动化未覆盖到

当前仍存在的问题

待解决
▲ 云端执行一次成功率 90% 左右，硬件运维及工具调用等稳定性还有一定的提升空间
▲ 单 case 执行耗时与 token 消耗仍需优化

通过就是真的通过，失败就是真的有问题：平台追求的不是通过率高，而是结论可信——宁可保守不给假绿灯。

回流：把这一轮跑出来的东西，接回下一轮

回流不是「跑完归档」，而是让本轮经确认的结论变成下一轮更聪明的起点——只回写已确认的，未确认的只留记录、不固化成规则。

回写「资产」· 沉淀下来的东西

回写「能力」· 让 agent 变聪明

1 知识库更新

· 确认的「变更后逻辑」回写进知识资产

· 场景规则、数据约束、接口断言、业务 Skill

· 下一轮需求分析、用例生成直接消费

2 用例更新

· 验证过的结论沉淀成回归资产

· 新增、更新回归用例，建线上流量回归标签

· 一次性执行 → 可持续的回归基线

3 流程 review · 复盘

让 agent 越跑越聪明

· 复盘反复出现的漏测、误判、执行限制

· 经人确认 → 补成「生成规则 / 反例」喂回 agent

· 下一轮生成、数据、断言因此更准

案例演示 · 真实导购交叉需求测试

经确认结论回流 → 下一轮输入（闭环）

1 源头校验 · 拦截不确定性

校验 PRD 完备性 · 方案可验证性 · 多轮澄清达成共识 | 人准入：确认需求范围

结构化交付包（固定 schema）

2 结构化收敛 · 生成自检用例 70 条（接口 52 + UI 18）

生成 + 三轮自检：范围 / 风险 / 数据 / 断言 无缺口；每条用例带数据绑定

结构化用例 + 数据绑定

3 规模化执行 · 产出可追溯证据 接口 50/52 · UI 17/18

断言独立于调用成功（治假绿灯）· 真机并发 · 自校正：翻案 / 撤销误报 / 两轮复跑

可追溯证据 + 已确认结论

4 证据与回流 · 经确认后回流 3 个 bug + 4 个标签

只回流已确认的变更后逻辑，未确认只留记录 | 人准出：判定验收

下一轮输入（回流资产）

放大看 ③ 规模化执行 · 两条真实留痕（接口自动化后台 · 真机执行流）

接口段 · 自动化任务与结构化断言

接口自动化平台 · 执行脚本

任务 T-20260916-012 · 承接页导流位写入 + DB 校验 · 环境 pre

步骤① 写入导流位（HSF）· 核对接口返回

```
{ "phase":"pre", "type":"hsf",
 "properties":{"service":"...LandingConfigService",
  "method":"saveCrossFlowRule","args":[{...}]},
 "assert":"result.success==true && result.code=='200'
   && result.data.ruleId != null"} ← 接口返回 √
```

步骤② DB 校验（DMS）· 核对字段落库

```
{ "phase":"pre", "type":"dms",
 "properties":{"appName":"F_CROSS_FLOW_APP",
  "logic":false,"script":"SELECT status, biz_type,
   start_time FROM cross_flow_rule
   WHERE rule_id=${ruleId}" },
 "assert":"success==true && items[0].rowCount==1
   && items[0].status=='ENABLE'
   && items[0].biz_type=='CROSS_FLOW'" }
```

接口 success / 200 √ · DB 记录存在 rowCount=1 √

items[0].status 期望 'ENABLE' · 实际 'DRAFT' ✗

字段级断言揪出：记录落了、状态没落对 → 规则不生效

接口报成功、行也在 ≠ 字段真写对

UI 段 · 真机执行流（按真实页面）

执行流 Job #22011 已完成 ▶ 回放

√ 验证通过 校验 2/2 通过

交叉导流承接页 · 不满足条件

不出目标卡 – 临近出发用户

安卓 · 32 轮 · 05:07 · 12 截图

› 用例定义

执行步骤 精简 23

① 环境准备 5 步

② 打开承接页 · 等首屏 1 步

③ 下滑浏览推荐卡区域 5 步

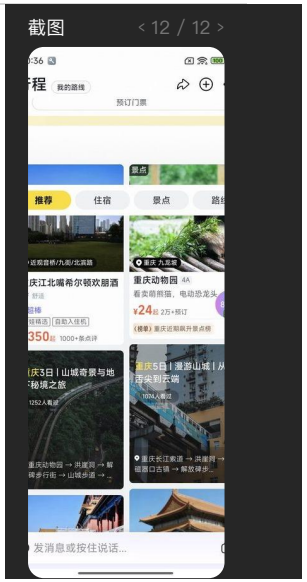
④ 验证检查点 · 提交结论 2 步

校验点（CHECKPOINT）

√ 不出「权益卡·提前预订」推荐卡 出发仅 3 天 < 「大于 3 天」门槛

√ 兜底卡「XX 提前订」照常渲染

截图 < 12 / 12 >



真机回放 · 12 / 12 帧

接口成功、记录也在，字段却没落对 → 假性成功

= 接口返回 + DB 字段级断言，才算验证到位

校验点逐条打勾 · 真机回放留痕

= 可追溯证据，结论可回放、可复现

真实案例时间

案例 A

社群新业务 · 材料全 AI 产出

阶段一 · 卡在这

用例生成 一天 → 20min，但材料全 AI 产出，生成采纳率只有 ~50%

阶段二 · 这样补

把校验前置到源头，用例生成消费的是已校验的结构化交付包

源头稳住了

澄清问题 ↓ 50%+

需求 case 采纳率 ~80% · 源头一次说清

案例 B

存量交叉推荐 · 历史逻辑重

阶段一 · 卡在这

历史逻辑重、存量规则多，单次改动影响面难评估；接口自动跑又「假性成功」

阶段二 · 这样补

存量一次梳理后，已确认结论回流自更新知识库 → 闭环；结构化断言 → 治「假性成功」

存量结论可信

发现 bug 20+

累计 178 个回归标签 · 133 条用例 AI 录入

案例 C

保险辅营发版回归 · 端侧 UI

阶段一 · 卡在这

本地 Agent 能自主跑真机，但单条慢（AI≥30min）、执行不稳、量上不去

阶段二 · 这样补

建飞碟平台：并发治慢 + 确定性工程化治稳 + 工具库交叉校验治准

规模化回归

7 个真实缺陷

300+ 条全量回归 · 6 批并行压到 1/3 时间

03

阶段三：AI 深入之后的新问题

执行交给 AI 后，人的判断力如何生长，面向人的体验如何保障

阶段三：AI 越深入，两个更根本的问题

问题一 · 人的能力如何生长

当「执行」交给 AI，质量同学的判断力和经验，从哪里生长?

· 判断力过去在一线执行里练出来；执行上移给 AI 后，它该沿什么新路径长?

· 经验怎么从「个人手感」沉淀成「可传承的资产」，又不丢掉手感本身?

· 人要守的是判断——那让「判断」保持敏锐，又该靠什么?

↔

问题二 · 体验与交互

体验、交互、易用这些 for 人的质量，怎么进 AI 的闭环，而不是被漏掉?

· 体验 / 交互 / 易用本质是 for 人——好不好用没标准答案，真实用户用了才知道

· 过去靠测试同学当第一个真实用户顺手发现；现在要不要把它正式接进来?

· AI 判得了「按钮在不在」，判不了「这个顺序让人烦」——这类判断该留给谁?

单点提效不是终点，可信闭环才是关键

从「每个环节更快」到「整条链路可信」——我们走过的三段路

源头更清楚

① 先校验需求与方案，再让 AI 生成与执行；缺口在源头显式列出，不靠下游猜测。

结论有依据

② 不把「跑通了」当「验证到了」：业务断言 + 真机证据 + 服务端对账，才能给一个 PASS。

经验能复用

③ 将本轮确认的业务规则、补齐的遗漏场景沉淀到知识库和用例中；下一轮少做重复梳理，少漏已遇到的问题。

让验证追上生成 —— 不只是生成更多用例、执行更快，而是让每次变更都能更高效地得到「有依据」的质量判断。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

让验证追上生成

Let Verification Catch Up With Generation

